SWE 4739
Embedded Software Development

Lecture 3
Secure Application Design

Sabrina Islam
Lecturer, CSE, IUT
Contact: +8801832239897
E-mail: sabrinaislam22@iut-dhaka.edu

Secure embedded software is important for a company:

- To protect intellectual property to prevent competitors from catching up or gaining a competitive edge.
- To protect users' data.
- To prevent high-profile attacks that reach the media having catastrophic consequences to a brand.

Platform Security Architecture (PSA)

PSA is designed to give developers holistic resources to simplify security.

PSA consists of

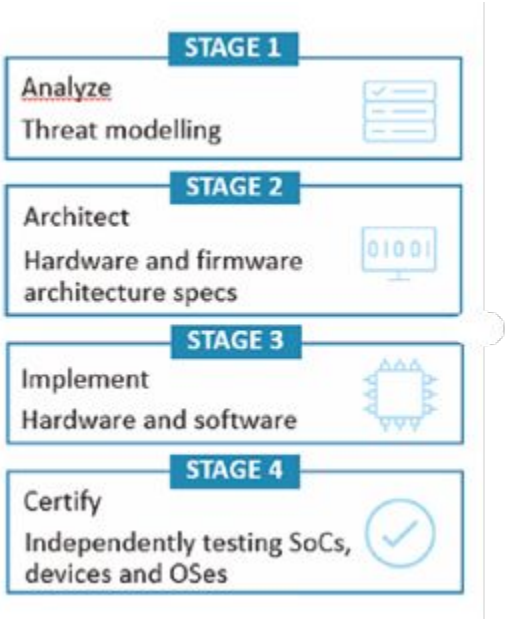
- A set of threat models and security analysis documentation.
- Hardware and firmware architecture specifications.
- An open source firmware reference implementation.
- An independent security evaluation scheme

Platform Security Architecture (PSA)

PSA includes four stages that can help guide a development team to constructing secure firmware. The four PSA stages include

- Analyze
  - analyze system requirements, identify data assets, and model the threats against the system and assets.
- Architect
  - developers select their microcontroller and associated security hardware in addition to architecting their software.
- Implement
  - developers can leverage PSA Certified components, processes, and software throughout the process to accelerate and improve software security.
- Certify
  - PSA Certified is a security certification scheme for Internet of Things (IoT) hardware, software, and devices.

Platform Security Architecture (PSA)



PSA Stage 1 – Analyzing a System for Threats and Vulnerabilities (TMSA)

- Identifying the assets that need to be protected and the threats those assets will face.
- Define security requirements to protect the assets.
- The security requirements will then dictate the security strategies and tactics used and the hardware and software needed to protect those assets adequately.

PSA Stage 1 – Analyzing a System for Threats and Vulnerabilities (TMSA)

TMSA Step 1: Identifying Assets

An IoT device commonly has data assets such as

- A unique device identification number
- The firmware
- Encryption keys
- Device keys
- Etc.

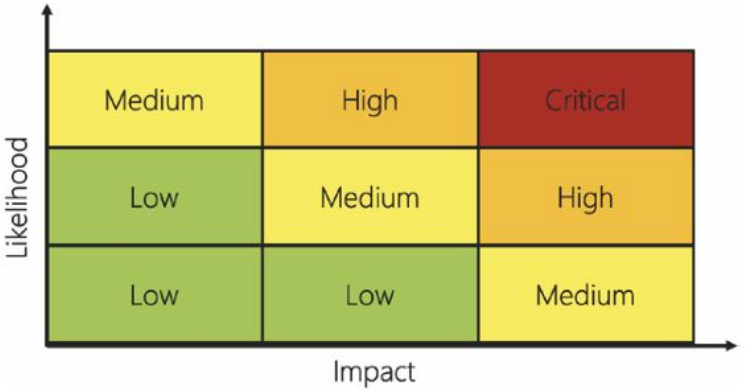
Every product will also have unique data assets that need to be protected. For example, a biomedical device might protect data such as: A user profile, Raw sensor data, Processed sensor data, etc.

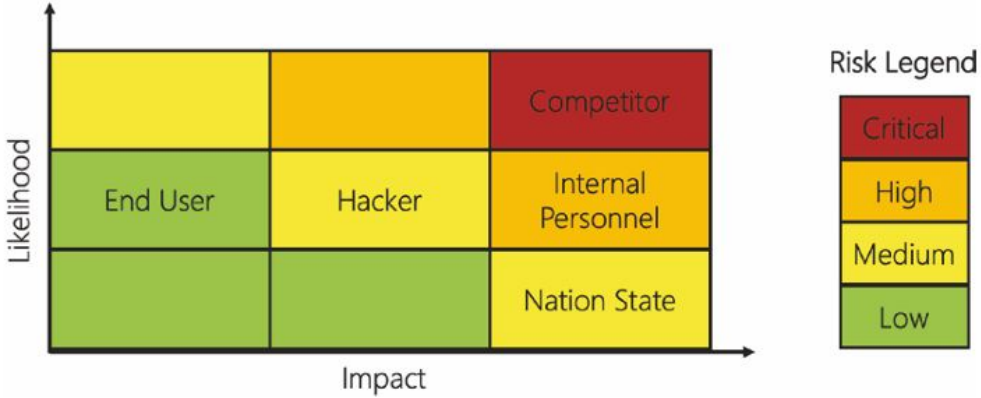
TMSA Step 2: Identifying Adversaries and Threats

Identifying adversaries can be as simple as creating a list. Developers should rank each adversary based on the risk (impact) to the business, user, and system and determine the likelihood of an attack.

Adversary	Impact	Likelihood	Comments
Competitor	High	High	IP theft
End user	Low	Medium	Feature availability
Hackers	Medium	Medium	Data theft, malware
Nation state	High	Low	Data theft, malware
Internal personnel	High	Medium	IP and data theft

TMSA Step 2: Identifying Adversaries and Threats





TMSA Step 3: Defining Security Objectives

1. Identifying which combination of confidentiality, integrity, and authenticity will protect the assets.
    a. Confidentiality, integrity, and authenticity are often called the CIA model.
    b. Confidentiality indicates that an asset needs to be kept private or secret. For example, data assets such as passwords, user data, and so forth would be confidential.
    c. Integrity indicates that the data asset needs to remain whole or unchanged. For example, a team would want to verify the integrity of their firmware during boot to ensure that it has not been changed.
    d. Authenticity indicates that we can verify where and from whom the data asset came.

TMSA Step 3: Defining Security Objectives

1. Identifying which combination of confidentiality, integrity, and authenticity will protect the assets.

Data Asset	CIA	Threat
Location	C, I	Man-in-the-middle, Repudiation, Impersonation, Disclosure
Raw Sensor Data	C, I	Tamper (Disclosure)
System Configuration	C, I	Tamper (Disclosure)
Credentials	C, I	Impersonation, Tamper (Disclosure), Man-in-the-middle, Escalation of Privilege (firmware abuse)
Firmware	C, I, A	Tamper, Escalation of Privilege, Denial of Service, Malware

TMSA Step 3: Defining Security Objectives

2. The security objective that will be used to secure the asset. The most common in an embedded system are

a. Access control
b. Secure storage
c. Firmware authenticity
d. Communication
e. Secure state

TMSA Step 4: Security Requirements

Security is not static throughout the product life cycle. The system may have different security requirements at various stages of development. As part of the security requirements, teams should outline how their requirements may change with time.

TMSA Step 5: Summary

The TMSA overview is often just a spreadsheet that summarizes the assets, threats, and how the assets will be protected. The detailed TMSA analysis document will walk through the following:

- Explain the target of evaluation
- Define the security problem
- Identify the threats
- Define organizational security policies
- Define security objectives
- Define security requirements

TMSA Step 5: Summary

This spreadsheet is an "at a glance" summary of a Threat Model and Security Analysis (TMSA) document (DEN0075). It is a quick-reference summary of assets, threats, impact and security requirements that are provided in Excel format to allow you to easily edit and extend. We hope that you find this document useful as a starting point.

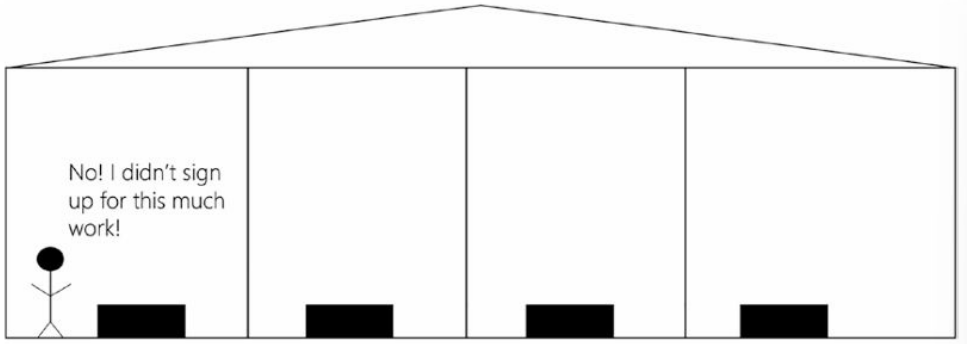
CI=Arm CryptoIsland, PSA=Arm Platform Security Architecture

Asset	Security Property	Threat	Entry Point of Threat (where the attack is launched from ex: malware, network, JTAG, etc)	Impact of Vulnerability	Severity (CVSS Rating)	Mitigation/Security Requirement	Arm's Technology
Firmware	Integrity	Tamper	Malware, bug, mass storage access, JTAG, network, update	Install malware	Critical: 9 CVSS:3.0/AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:H	Support secure boot flows (authenticate firmware)	[CI] Loaded SW validation functionality
		Escalation of privilege (Firmware Abuse)		Launch DDoS		Enforce principle of least privilege	
						Support secure firmware update (authenticate update)	[CI] SW update validation
							[PSA] Trusted Boot features
		DoS (Firmware Abuse)		Tamper/Steal location		Support anti-rollback of firmware	[CI] SW update validation
				Permanent bricking of device			[PSA] Firmware Update features

PSA Stage 2 – Architect

Security Through Isolation

- Isolation is the foundation on which security is built into an application.
- The isolation acts as a barrier limiting the attack surface that an attacker can leverage to access an asset of interest on the device.



PSA Stage 2 – Architect

Security Through Isolation

- Break the application into separate tasks, processes, and memory locations.
- Each of them will have limited access to the data assets and system resources.
- The underlying hardware must support hardware-based isolation.

PSA Stage 2 – Architect

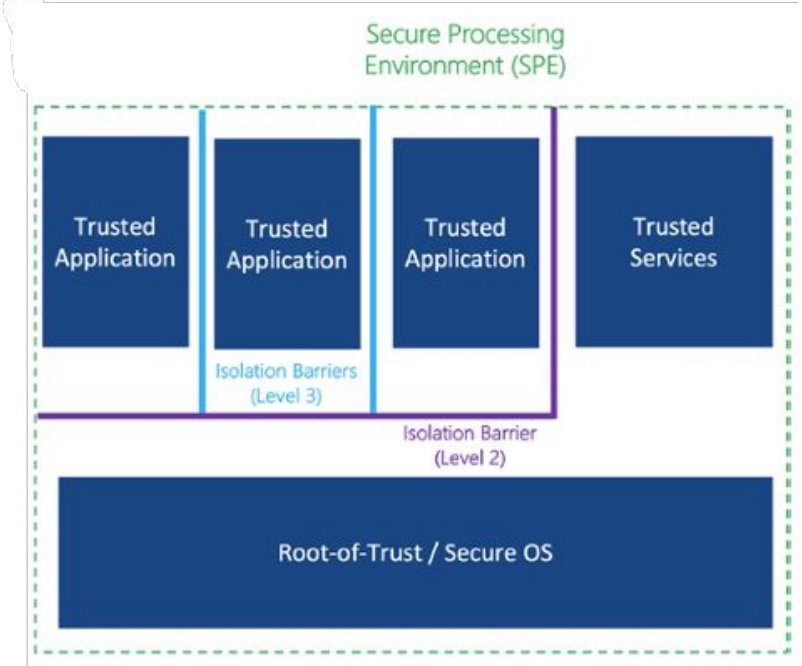
Security Through Isolation

There are three levels of isolation that designers need to consider:

- Processing environments,
- Root-of-Trust with trusted services,
- Trusted applications.

PSA Stage 2 – Architect

Security Through Isolation



Isolation Level 1: Processing Environments

The system is divided into two hardware-isolated environments:

- Secure Processing Environment (SPE):
  - Also called Trusted Execution Environment (TEE)
- Nonsecure Processing Environment (NSPE)
  - Also called Rich Execution Environment (REE)

Isolation Level 1: Processing Environments

Secure Processing Environment (SPE)

- Contains:
  - Root-of-Trust (RoT)
  - Cryptographic services
  - Trusted application components
- Handles sensitive and security-critical operations

Nonsecure Processing Environment (NSPE / REE)

- Contains:
  - RTOS (or general OS)
  - Regular application code
- Used for normal application development and execution

Isolation Level 1: Processing Environments

A designer can use two mechanisms to create the first level of isolation:

- Multicore processors
- Arm TrustZone

Isolation Level 1: Processing Environments

Using Multicore Processors

- One of the processing cores in the multicore solution to act as the NSPE
- The second core will be used to serve as the SPE
- Cores communicate using:
  - Shared memory
  - Interprocessor communication (IPC)
- Secure core exposes only limited services to NSPE
- Advantage: Both environments can run simultaneously

Isolation Level 1: Processing Environments

Using Arm TrustZone

- Hardware-based isolation in one processor
- Processor switches between NSPE and SPE
- Switching takes only 3 clock cycles
- When the SPE is complete, the processor switches back to the NSPE.
- More cost-efficient than multicore

Isolation Level 2: Root-of-Trust and Trusted Services

Root-of-Trust (RoT)

- The trust anchor of the system
- Located inside the Secure Processing Environment (SPE)
- Supports secure boot and security services
- Must be hardware-based and immutable (cannot be modified)
- Resistant to attacks
- A Root-of-Trust Typically Includes hardware-accelerated cryptography, true Random Number Generator (TRNG) and secure storage

Isolation Level 2: Root-of-Trust and Trusted Services

Trusted Services

- Also Inside SPE
- Trusted services run on top of the Root-of-Trust.
- Common examples:
  - Secure boot (chain of trust)
  - Device provisioning
  - Device attestation
  - Secure storage
  - Transport Layer Security (TLS)
  - Firmware Over-the-Air (FOTA) updates

Isolation Level 3: Trusted Applications

- Developer-created applications
- Run inside the Secure Processing Environment (SPE)
- Execute in their own isolated memory space
- Provide services that require: Secure data access and secure hardware resource access
- Trusted application is built upon the Root-of-Trust and other trusted services.
- Often implemented by leveraging the capabilities provided by an open source framework called Trusted Firmware M (TF-M).
- TF-M Provides:
  - Secure boot
  - Isolation management
  - Secure storage and attestation services

PSA Stage 2 – Architect

Architecting a Secure Application

Architecting a secure application requires the developer to create layers of isolation. Embedded software architects should create a model of their secure application which is thought through carefully and diagrammed. The model should

- Identify what goes in each processing environment
- Break up the system into trusted applications
- Define the RoT and trusted services
- Identify general mechanisms for protection such as MPUs, SMPUs, etc.
- Define general processes and memory organization (in a generic way)

PSA Stage 2 – Architect

Architecting a Secure Application

There are three different types of threads that architects and developers should consider:

- Nonsecure threads only call functions that exist within the NSPE.
- Secure threads which only call functions that exist within the SPE.
- Hybrid threads execute functions within both the SPE and the NSPE. Hybrid threads often need access to an asset the architect is trying to protect.

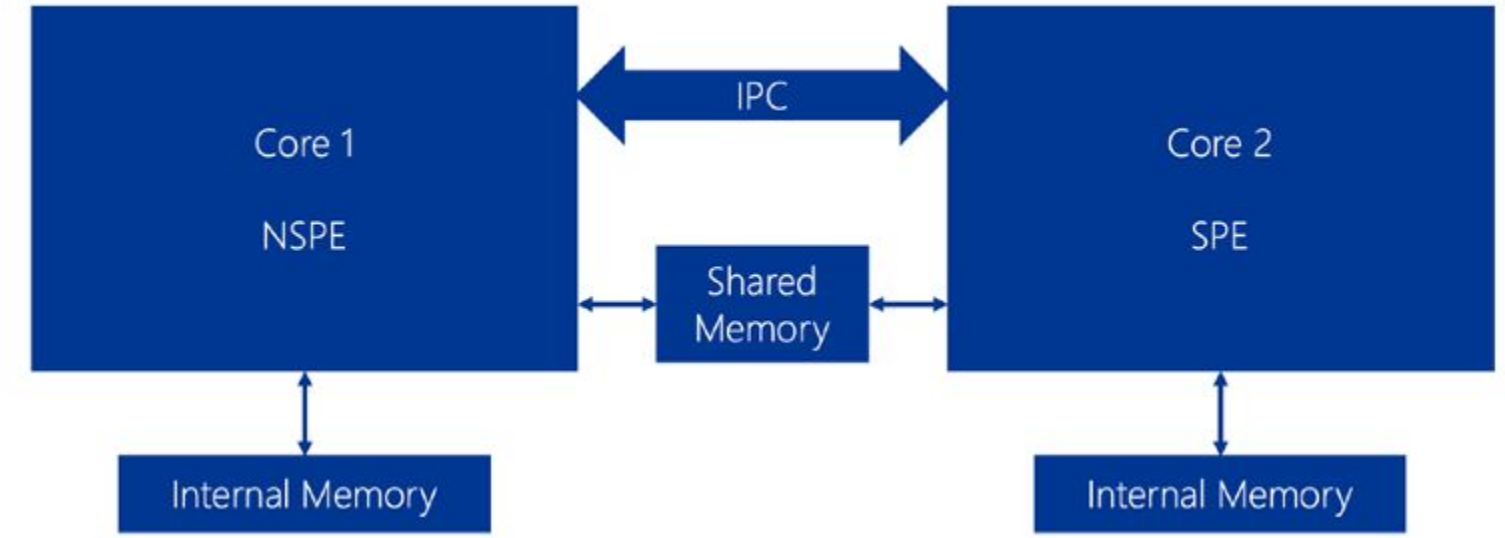
PSA Stage 3 – Implementation

Multicore Processors

- Usually dual-core and cores may or may not be the same processor architecture.
- Advantages
  - True hardware isolation
  - Parallel execution (better performance)
  - Reduced attack surface (no external security chip needed)
  - Each core is isolated so harder to exploit
- Disadvantages
  - Higher cost (often more expensive than single-core)
  - Increased software complexity
  - Harder debugging
  - Possible performance bottlenecks if hardware is not selected properly

PSA Stage 3 – Implementation

Multicore Processors



PSA Stage 3 – Implementation

Single-Core Processors with TrustZone

- TrustZone allows developers to divide:
  - Flash: NSPE or SPE
  - RAM: NSPE or SPE
  - Peripherals: NSPE or SPE
- CMSIS-Zone Support
  - Helps configure and partition memory, peripherals, system resources
  - Requires one project for NSPE and one project for SPE

PSA Stage 3 – Implementation

Single-Core Processors with TrustZone

- Execution Behavior
  - Only one environment runs at a time
  - Switching from NSPE to SPE takes deterministic 3 clock cycles
  - After secure operation:
    - CPU registers are scrubbed (partially)
    - Processor switches back to NSPE
- Boot Process in TrustZone
  - System boots in SPE first
  - Secure bootloader:
    - Validates NSPE firmware
    - Then jumps to NSPE for normal execution

PSA Stage 3 – Implementation

Single-Core Processors with TrustZone

- Secure Gateway Concept
  - NSPE cannot directly access SPE memory. NSPE can only call exposed secure functions. These entry points are called secure gateways
- Advantages
  - Lower cost than multicore
  - Hardware-enforced isolation
  - No external security processor needed
- Disadvantages
  - No simultaneous execution
  - Small clock-cycle overhead during switching
  - Entire memory map exists on one core: larger attack impact if compromised
  - Requires careful memory configuration

PSA Stage 3 – Implementation

Chain-of-Trust

Developing a Chain-of-Trust involves several steps for an embedded application.

1. Root-of-Trust (RoT)

- Hardware-based (recommended)
- Trust anchor of the system
- Often provisioned by chip manufacturer
- Developer later adds company keys and credentials

2. RoT Verifies Secure Bootloader

- Uses cryptographic hash/signature
- Validates: Integrity and Authenticity
- If valid then launches secure bootloader

PSA Stage 3 – Implementation

Chain-of-Trust

3. Secure Bootloader Responsibilities

- Checks for firmware updates
- If update requested, performs firmware update. If not then continues normal boot
- Validates:
  - NSPE application firmware
  - Secure firmware (SPE)
- Bootloader does NOT directly launch NSPE application

4. Secure Firmware (SPE)

- Secure runtime environment
- Launches NSPE only after system is validated and the security checks pass
- Often based on trusted Firmware-M

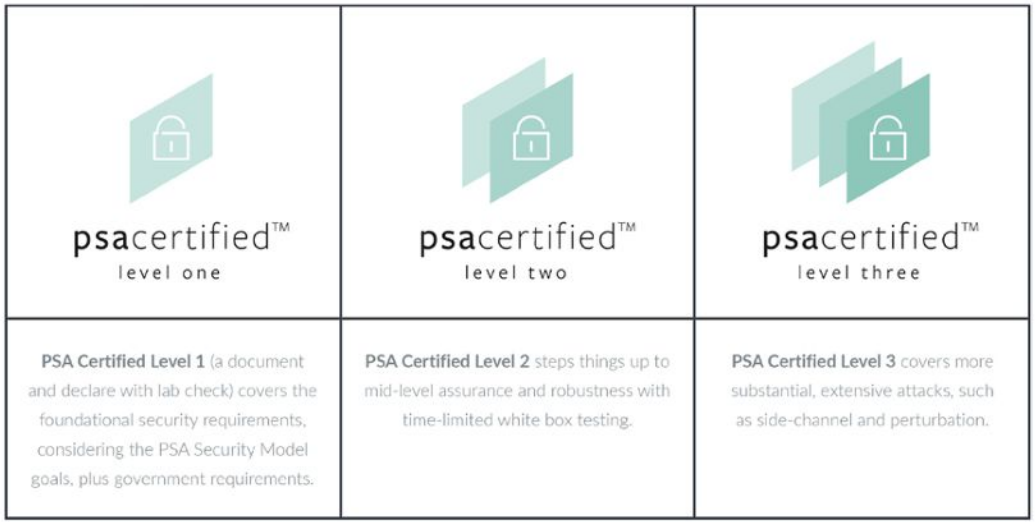
PSA Stage 4 – Certify

- Ensures system meets security requirements
- Verifies use of industry best practices
- Provides independent security validation
- Recommended even if full certification is not pursued. At least penetration testing should be performed.
- PSA Certification
    - Independent security evaluation scheme
    - Designed for: IoT chips, Operating systems, Devices

PSA Stage 4 – Certify

PSA Certified Levels

- Three certification levels
- Each level increases security robustness and requires stronger evaluation
- Higher level means stronger protection against advanced attacks



Reference:

1. Embedded Software Design: A Practical Approach to Architecture, Processes, and Coding Technique - Jacob Beningo

Thank You!